

packingSDF — Input-Agnostic SDF + Disk Packing Module

A concise, implementer-focused specification for an **input-agnostic** `packingSdf` module. This document is intentionally focused on how the module **processes data**: what it needs (inputs / preconditions), the internal processing flow, and exactly what it produces (outputs / data contracts). The goal is for a developer to implement the module directly from this description.

1 Goals and scope

Primary goal: Provide an encapsulated module that consumes *any* 2D input (polygons, triangles, raster/alpha bitmaps, or video frame buffers) and produces:

- a **signed distance field (SDF)** over a configurable grid (with narrow-band and multiresolution options);
- a **disk-packed seed set** (maximal or Poisson-like) suitable for seeding, remeshing, sampling, or LOD;
- derived maps (normals, curvature proxy) and stable, typed-array outputs and sampling helpers for downstream systems.

Design constraints: - Input-agnostic: unify disparate inputs early by producing a canonical mask/coverage grid. - Encapsulated API: single class with clear methods for single-shot and streaming (video) use. - Deterministic default behavior with optional stochastic algorithms (Poisson).

2 Terminology / nomenclature

- **Grid:** regular rectangular grid of `w × h` cells. Each cell corresponds to a world-space coordinate via `(origin, dx, dy)`.
- **mask:** boolean inside/outside raster produced from input (0 outside, 1 inside) or coverage `[0..1]`.
- **edt_in / edt_out:** Euclidean Distance Transform results computed on `mask` and inverted `mask` respectively.
- **signed_sdf:** scalar per-grid-cell; negative values inside (convention) and positive outside.
`signed_sdf = mask ? -edt_in : edt_out`.
- **narrow-band:** region where `|signed_sdf| ≤ band` (band in cell units). Only full distances need to be kept in band.
- **disk:** `{x, y, r, score, seedId}` — center coordinates in world space and radius in world units.

3 What the module needs (inputs / preconditions)

Provide the following to get predictable, aligned outputs:

1. **Coordinate frame / domain box** (required):

2. Either `{origin:[x,y], dx, dy, w, h}` or `{bbox:[xmin,xmax,ymin,ymax], w, h}`.
3. This maps grid cell indices to world coordinates and must match the caller's domain for consistent sampling.

4. **Input representation** (one of):

5. `polygon`: `Array<[x,y]>` (closed winding assumed).
6. `triangles`: `Array<[[x,y],[x,y],[x,y]]>` (2D triangles in the chosen domain).
7. `bitmap` / `alpha`: raw pixels, `ImageBitmap`, `Canvas`, or typed array plus width/height and optional transform to world coords.
8. `frameBuffer` / video frame: `ImageBitmap` or pixel buffer for streaming.

9. **Optional hints / parameters:**

10. `insideThreshold` for coverage → boolean mask conversion.
11. `band` (narrow-band half-width in cells).
12. `rMin`, `rMax` for disk radius clamping.
13. `packingEps` to inflate mask during greedy packing.
14. `device` preference: `'cpu'` or `'gpu'`.
15. For streaming: motion vectors or optical-flow buffers (optional) to enable warp-warm starts.

16. **Orientation / projection info (optional but recommended):**

17. If supplying triangles from 3D geometry, provide a 2D projection (UVs) or indicate the projection method used.
18. If triangle winding is inconsistent, the module will rasterize and determine sign by flood-fill.

4 What the module produces (outputs / data contracts)

All outputs are typed and indexable. The module returns a single result object and exposes sampling helpers.

Primary return object (fields and types):

```
{
  grid: {w:int, h:int, dx:float, dy:float, origin:[x,y]},
  signed_sdf: Float32Array(w*h), // world units
  sdf_in: Float32Array(w*h), // optional, distance-to-boundary inside
  sdf_out: Float32Array(w*h), // optional, distance-to-boundary outside
  narrowMask: Uint8Array(w*h), // 1 where |signed_sdf| <= band
  normals: Float32Array(2*w*h), // gx,gy per cell (not always normalized)
  curvature: Float32Array(w*h), // Laplacian proxy per cell
  densityMap: Float32Array(w*h), // optional target edge length per cell
  disks: Array<{x:float,y:float,r:float,score:float,seedId:int}>,
}
```

```
meta: {timestamp:number, device:'cpu'|'gpu', params:Object}
}
```

Indexing convention: cell index $i = y*w + x$. Cell center world coordinate: $wx = origin.x + (x + 0.5) * dx$.

Sampling helpers: - `sampleSdfAt(x,y)` → scalar (bilinear sample) - `sampleNormalAt(x,y)` → `[nx,ny]` (interpolated) - `sampleCurvatureAt(x,y)` → scalar - `sampleRegion(bbox)` → typed arrays for that bbox aligned to the module grid

Transfer formats: prefer typed arrays for CPU workflows. For GPU-first flows the module can optionally provide GPU texture references or raw RGBA float exports.

5 Processing pipeline — step-by-step (how data is transformed)

This section describes the canonical internal flow from input → outputs.

1. Input normalization (rasterization / mask creation)

2. Accepts polygon / triangles / bitmap / frame buffer.
3. Produces `mask` (Uint8Array or Float32Array coverage) at configured `w*h`.
4. Methods:
 - Polygon: scanline fill with optional supersampling for anti-aliasing.
 - Triangles: barycentric rasterization (CPU or GPU).
 - Bitmap: threshold alpha or use coverage directly.

5. Result: `mask[w*h]` (values in {0,1} or [0..1] if coverage).

6. Boundary extraction & prepared arrays

7. From mask compute a binary `boundary` indicator (edge pixels) for EDT initialization.
8. Prepare `f[i]` arrays for EDT where `f[i] = 0` for boundary, `INF` elsewhere.

9. Distance Transform (EDT)

10. CPU: Felzenszwalb's two-pass EDT (linear time $O(w*h)$). This produces distances in cell units; multiply by `dx` / `dy` to get world units.
11. GPU: JFA or compute-shader EDT for approximate or fast results.
12. Compute `edt_in` on `mask` and `edt_out` on inverted `mask`.

13. Signed SDF assembly

14. For each cell `i`:
 - If `mask[i] > insideThreshold` then `signed_sdf[i] = -edt_in[i]` else `signed_sdf[i] = edt_out[i]`.

15. Apply optional clamping to `|signed_sdf| <= clampFar` and build `narrowMask` where `|signed_sdf| <= band`.

16. Derivatives (optional but commonly produced)

17. Compute central-difference gradients `(gx, gy)` on `signed_sdf` producing `normals` buffer.

18. Compute discrete Laplacian as a curvature proxy.

19. Optionally smooth SDF before derivatives to reduce noise.

20. Density mapping (optional)

21. Map `s = |signed_sdf|` to a target length `l = clamp(k * s + l_min, l_min, l_max)` producing `densityMap`.

22. Disk packing / seeding

23. Greedy maximal (grid-based): build max-heap of cells by `|signed_sdf|` and iteratively place disks; mask out neighborhoods.

24. Poisson-guided: compute local radius `r(x)` from `densityMap` and run Bridson's Poisson-disk with spatial hashing.

25. Continuous maximal: fit maximal inscribed circles using continuous queries (heavier).

26. Output: `disks[]` array of `{x, y, r, ...}` in world units.

27. Export / sampling helpers

28. Expose typed arrays and sampling methods for callers.

29. For streaming/video, provide `updateRegion` information describing which cells changed.

6 Streaming / video-specific behavior

The module supports streaming workflows and provides incremental update strategies:

- **Full recompute:** rasterize + EDT + packing every frame (simple, expensive).
- **Motion-warp warm-start:** if motion vectors are supplied, warp previous `signed_sdf` and disks to approximate the new frame; recompute EDT only in delta regions.
- **Delta-mask narrow-band re-EDT:** compute `diff = xor(prevMask, currMask)`, dilate by `R`, and run EDT only inside `updateRegion`. Merge partial EDT with prior field outside region.
- **Disk propagation + local reseed:** keep disks whose centers still fall in valid areas; reseed locally where coverage changed. Use hysteresis to avoid flicker.

The module returns `meta.updateRegion` (bbox or mask) for callers to cheaply determine what downstream processing must be redone.

7 Algorithms — notes & implementation choices

- **EDT (CPU):** use Felzenszwalb (exact, linear), returns distances in cells — convert to world units.
- **EDT (GPU):** JFA for speed; optionally refine results.

- **Rasterization:** CPU scanline / triangle rasterizer for deterministic behavior; GPU rasterization for speed.
- **Packing:** greedy grid-based is simplest and deterministic; Poisson gives blue-noise but is heavier.
- **Derivatives:** central differences are straightforward; consider smoothing to reduce noise.

Performance tradeoffs and recommended defaults: - Grid sizes: `512×512` or `1024×1024` are common. Choose depending on smallest feature to resolve. - Narrow-band `band`: 8-16 cells is a practical default. - For real-time video use GPU pipeline or incremental narrow-band updates.

8 Public API (concise)

```
class PackingSdf {
  constructor(opts)
  setGrid({w,h,origin,dx,dy})
  computeAll(input, opts)      // one-shot compute -> result object
  updateFrame(frame, opts)     // streaming: returns partial result +
  updateRegion

  // sampling helpers
  sampleSdfAt(x,y)
  sampleNormalAt(x,y)
  sampleCurvatureAt(x,y)
  sampleRegion(bbox)

  // outputs
  getResult()    // returns last full result object
  getDisks()
}
```

Important

options:

```
{band, insideThreshold, clampFar, rMin, rMax, packingEps, device,
deterministicSeed}.
```

9 Data formats & examples

World → grid mapping

- Given `origin = [ox,oy]`, `dx,dy`, cell `(ix,iy)` maps to world point `x = ox + (ix + 0.5) * dx`.
- To sample an arbitrary world point `p`, compute fractional cell coords and bilinearly interpolate surrounding cells.

Example call (single frame)

```
const ps = new PackingSdf({w:512,h:512,origin:[0,0],dx:0.01,dy:0.01,band:
12});
const result = ps.computeAll({triangles: triangles2d}, {rMin:0.005, rMax:
```

```
0.1});
// result.signed_sdf, result.disks, result.densityMap available as typed
arrays
```

Example streaming sequence (video)

```
// initialization
ps.setGrid(grid);
let prev = null;
for each frame {
  const partial = ps.updateFrame(frameBitmap, {motionVectors});
  // partial.meta.updateRegion tells which cells changed
  // re-use prev fields outside updateRegion
  prev = partial;
}
```

10 Minimal skeleton (JS/TypeScript)

```
export class PackingSdf {
  constructor({w=512,h=512,origin=[0,0],dx=1,dy=1,band=16,device='cpu'}={}){
    this.setGrid({w,h,origin,dx,dy});
    this.band = band; this.device = device;
    this._alloc();
  }

  setGrid({w,h,origin,dx,dy}){ /* allocate buffers */ }

  computeAll(input, opts={}){
    const mask = this._rasterize(input);
    const edt_in = this._edt(mask,true);
    const edt_out = this._edt(mask,false);
    this._assembleSignedSdf(mask,edt_in,edt_out);
    this._computeDerivatives();
    this._packDisks();
    return this.result();
  }

  updateFrame(frame, opts={}){ /* motion warp + partial EDT + local packing
  */ }

  sampleSdfAt(x,y){ /* map world->cell and bilinear sample */ }

  getDisks(){ return this.disks; }
}
```

11 File layout suggestion

```
packing_sdf/  
  src/  
    index.js (exports PackingSdf)  
    rasterizer.js  
    edt_cpu.js  
    edt_gpu.glsl (or js wrappers)  
    packing_greedy.js  
    packing_poisson.js  
    temporal_updater.js  
    utils.js  
  test/  
    test_rasterizer.js  
    test_edt.js  
    test_packing.js  
  examples/  
    single_frame_demo.html  
    video_stream_demo.html
```

End of amended specification.

I removed any discussion specific to integration with external triangulation systems and focused the document strictly on module inputs, internal processing, outputs, streaming behavior, and implementer-facing APIs. If you'd like, I can now:

- Export this document as a downloadable markdown file and create a zipped package with a reference implementation, or
- Generate the CPU reference implementation (Felzenszwalb EDT + greedy packer) ready to paste into your repo.